Laboratory experiment
Implementation of a heart rate monitor

Group No. B3

Date 14/05/2026

Last Name/First Name	Student ID
Tufano Gaetano	2103199
Lucchetta Lucia	2112494
Chirpii Gabriel	2101350

Purpose of the exercise: to make a heart rate monitor and pulse oximeter based on infrared LED, red LED, photodiode, analog signal conditioning circuit, microcontroller board and display.

Instrumentation needed:

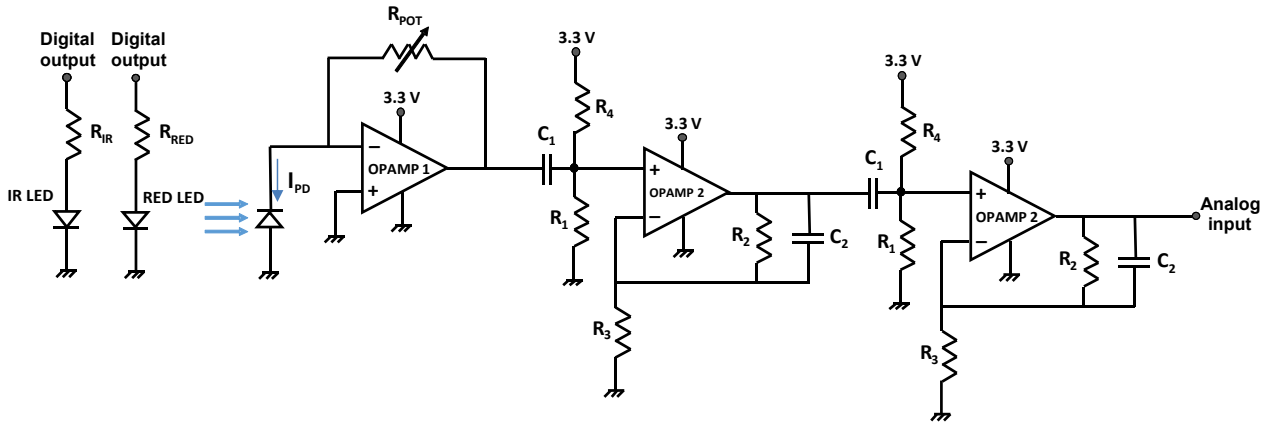
- Microcontroller board
- Breadboard
- Display
- Electronic components, see table below

Components needed:

Component type	Manufacturer Code/Value
Infrared LED, λ=850 nm SMT	Osram Opto, SFH 4250S
Red LED, λ=630 nm SMT	Osram Opto, LH T674
IR Photodiode/Visible SMT	Vishay VBP104S
Rail-to-rail operational amplifier	MCP6002
Resistances	To be calculated
Capacitor C_1	10 µF
Capacitor C_2	1 µF
Potentiometer R_{POT}	100 KΩ
TFT display	HX8357
Microcontroller board	
Breadboard and cables	

The circuit is powered through the USB port of the pc.

Circuit schematic



First experiment: PRELAB

1. Define the values of the resistors R_{IR} and R_{RED} so that currents equal to I_{RED} =15 mA and I_{IR} =3 mA, respectively, are obtained on the two red and infrared LEDs (use high current pins on microcontroller board, or add transistors to control the current)

   From the datasheet we found: $V_{f,red} = 1.75V$ so $R_{red} = (V_{cc} - V_{f,red})/I_{red} \approx 103.3\Omega$.
   We choose 100 Ω.
   From the datasheet we found: $V_{f,ir} = 2.6V$ so $R_{ir} = (V_{cc} - V_{f,ir})/I_{ir} \approx 233.3\Omega$.
   We choose 220 Ω.

2. Calculate the transfer function of the high-pass filter consisting of C_1, R_1, R_4

$$H_{HP(s)} = \frac{[s \cdot C1 \cdot (R1 \,||R4)]}{[1 + s \cdot C1 \cdot (R1 \,||R4)]}$$

3. Calculate the transfer function of the low-pass filter consisting of R_2, R_3, C_2

$$H_{LP(s)} = 1 + \left\{\frac{R2}{[R3 \cdot (1 + s \cdot R2 \cdot C2)]}\right\}$$

4. Calculate the overall transfer function of each bandpass filter

$$H_{BP(s)} = \left\{\frac{[s \cdot C1 \cdot Req]}{[1 + s \cdot C1 \cdot Req]}\right\} \cdot \left\{1 + \left[\frac{R2}{(R3 \cdot (1 + s \cdot R2 \cdot C2))}\right]\right\}$$

   Con $$Req = \frac{(R1 \cdot R4)}{(R1 + R4)}$$

5. Define the values of the resistors R_1, R_2, R_3, R_4 to obtain:

   a. That the high-pass filter consisting of C_1, R_1, R_4 has a cut-off frequency of 0.8 Hz
   b. That the rest dc voltage at the terminals v_+ is equal to 0.15 V
   c. That the low-pass filter consisting of R_2, R_3, C_2 has a cutoff frequency of 3 Hz
   d. That the dc gain of the low-pass filter consisting of R_2 , R_3 , C_2 is equal to 11

$$V_{bias} = 3.3 \cdot \left[\frac{R1}{(R1 + R4)}\right] = 0.15 \Rightarrow R4 = 21 \cdot R1$$

$$Req = \frac{1}{(2 \cdot \pi \cdot f_{HP} \cdot C1)} = \frac{1}{(2 \cdot \pi \cdot 0.8 \cdot 10^{-5})} \approx 19894\,\Omega$$

$$Req = R1 \,||\, R4 = (21/22) \cdot R1$$

$$R1 = 19894 \cdot \left(\frac{22}{21}\right) \approx 20842\,\Omega \Rightarrow \boldsymbol{chosen\ standard\ value}: \mathbf{22\ k\Omega}$$

$$R4 = 21 \cdot 20842 \approx 437682\,\Omega \Rightarrow \boldsymbol{chosen\ standard\ value}: \mathbf{470\ k\Omega}$$

$$R2 = \frac{1}{(2 \cdot \pi \cdot f_{LP} \cdot C2)} = \frac{1}{(2 \cdot \pi \cdot 3 \cdot 10^{-6})} \approx 53051\,\Omega$$

$$\Rightarrow \boldsymbol{chosen\ standard\ value}: \mathbf{56\ k\Omega}$$

$$Av = 1 + (R2 / R3) = 11 => R2 / R3 = 10$$

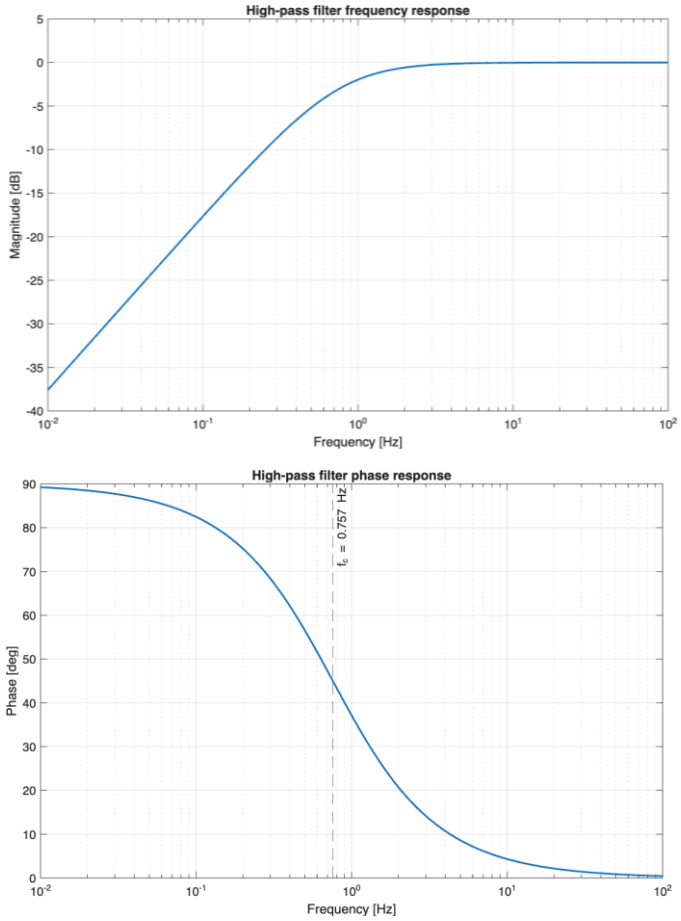
$$R3 = R2 / 10 = 53051 / 10 \approx 5305\,\Omega => \boldsymbol{chosen\ standard\ value}: \mathbf{5.6\ k\Omega}$$

6. Choose the nearest commercial value for the resistors R_1, R_2, R_3, and R_4

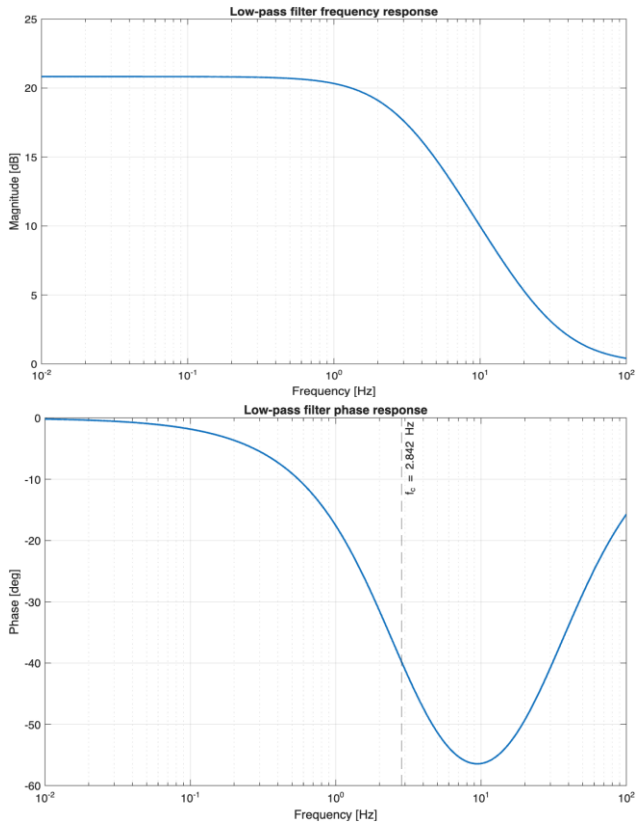
Already specified in the previous request.

7. With the values chosen in step 6, plot the frequency response

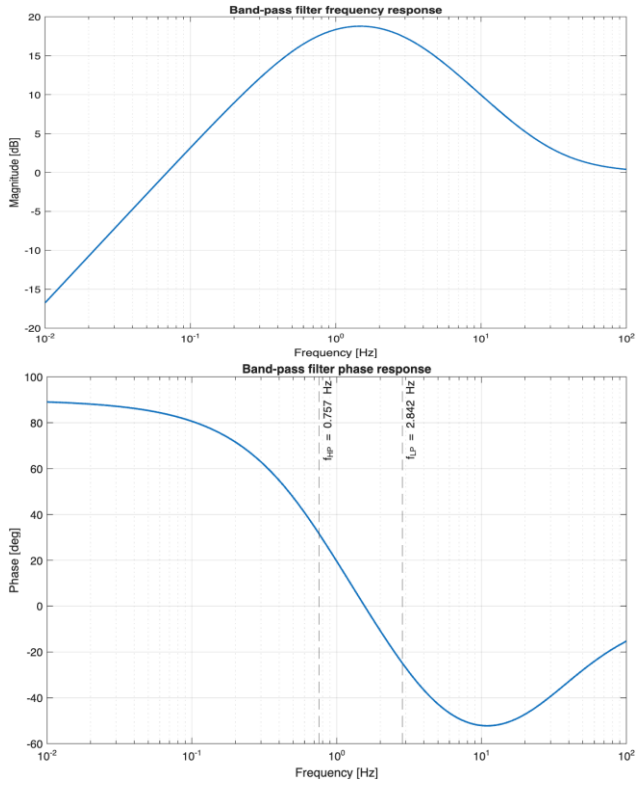
   a. of the high-pass filter consisting of C_1, R_1, R_4



b. of the low-pass filter consisting of R_2, R_3, C_2



c. of (one of) the bandpass filters



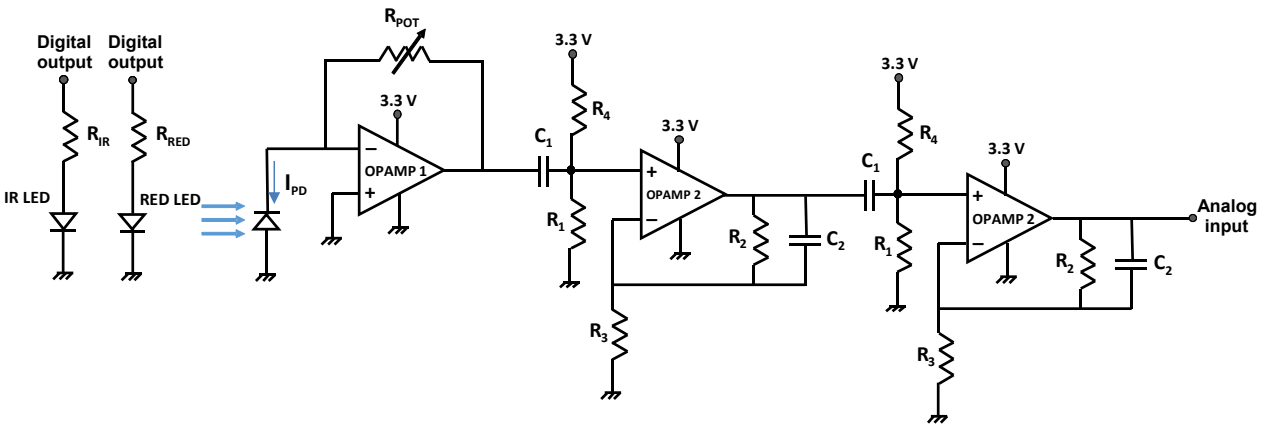
Add to the report the cut-off frequencies obtained using the commercial values of the resistors (which will be different from 0.8 Hz and 3 Hz)

Fc_HP = 1/(2*pi*Req_HP*C1) = 0.7573 Hz;

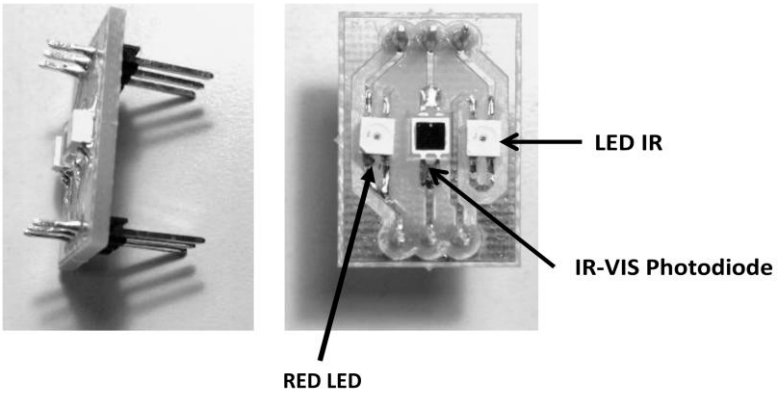
Fc_LP = 1/(2*pi*R2*C2) = 2.8421 Hz;

First experiment: In the laboratory

1. Mount the circuit show below, and connect the V_o signal to one of the analog inputs of the microcontroller board

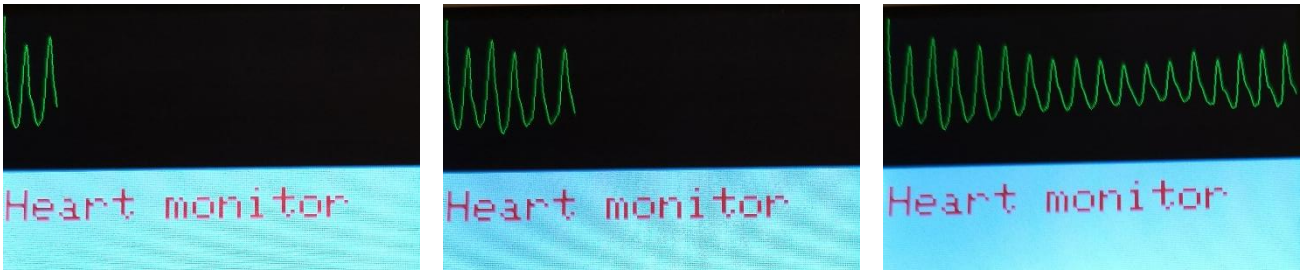


Use the custom board containing VIS/IR photodiode and IR LED diode shown in the figure below. Refer to the component datasheets to define the polarity of the components



2. Write a code to capture the signal at the output of the heart rate monitor, graphing it on the TFT display at the same time (see example in figure). Use the infrared LED for the time being for this analysis

   Hints: acquire a value every 20-30 ms, place it in an array (equal in size to the width of the screen), use the function *tft.drawLine(x-1, y(x-1), x, y(x), color)* to draw the segment corresponding to the last pair of data acquired. The input values to the ADC (0 to 1023) should be re-scaled so that they can all be represented on the display



Increasing time

3. Once the Array of values has been acquired, stop the acquisition. Write a function that will re-scale the array so that it is displayed on the top half of the screen, along with any other writing (see example in figure); report the corresponding code below

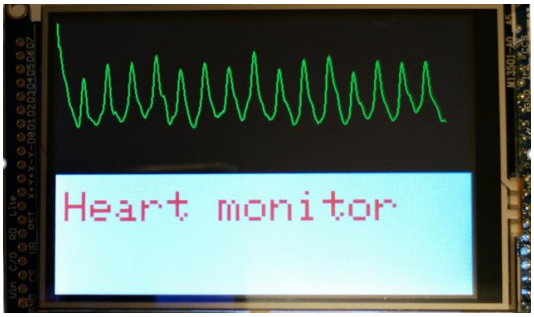
Hint: divide the code into functions as follows:

-function findmax(int data[])→ calculates the maximum value (in y) of the array data[]

-function findmin(int data[])→ calculates the minimum value (in y) of the array data[]

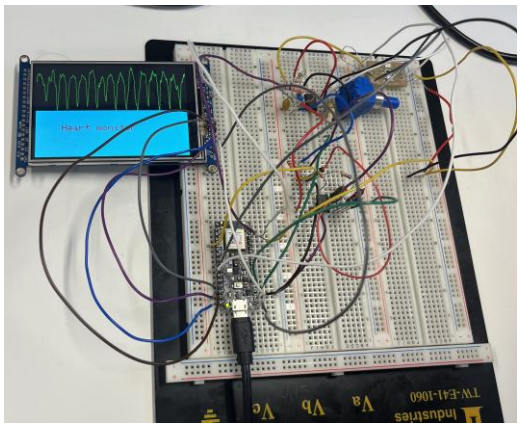
-function rescalearray(int data[])→ rescales the array data[] (via the map function) so that it occupies half the screen

-function plotarray(int data[])→ draws the plot of the array data[] on the display



.

Also report picture of the working circuit and related screenshots.



Code (**contains also request number 2**):

```
#include <Adafruit_GFX.h>
#include <Adafruit_HX8357.h>
#include <SPI.h>

// Define the pins used to control the Adafruit display
#define TFT_CS   10
#define TFT_DC    9
#define TFT_RST   -1

#define ANALOG_PIN A0      // Analog output of the circuit
#define IR_LED_PIN 7       // IR LED control pin

// Create an instance of the Adafruit_HX8357 display object
Adafruit_HX8357 tft = Adafruit_HX8357(TFT_CS, TFT_DC, TFT_RST);

// Define display parameters
#define SCREEN_WIDTH   480
#define SCREEN_HEIGHT  320

#define GRAPH_TOP      0
#define GRAPH_HEIGHT   160     // Upper half of the screen

#define TEXT_TOP       210

// The number of samples is equal to the screen width
#define N_SAMPLES SCREEN_WIDTH

// Time interval between two samples, in milliseconds
const int sampleDelay = 25;

// Delay between the final plot and the start of a new acquisition
const int pauseAfterPlot = 5000;

// Array used to store the ADC values
int data[N_SAMPLES];

// Array used to store the scaled y-coordinates for plotting the signal
int scaledData[N_SAMPLES];

int x = 0;
int previousY = 0;

// Function prototypes
int findmax(int data[]);
int findmin(int data[]);
void rescalearray(int data[]);
void plotarray(int data[]);
```

```
48. void drawText();
49.
50. void setup() {
51.   // Turn on the IR LED
52.   pinMode(IR_LED_PIN, OUTPUT);
53.   digitalWrite(IR_LED_PIN, HIGH);
54.
55.   // here we configure the analog input pin
56.   pinMode(ANALOG_PIN, INPUT);
57.
58.   // here we initialize the display
59.   tft.begin();
60.   tft.setRotation(1);
61.
62.   tft.fillScreen(HX8357_BLACK);
63.
64.   // here we initialize the data arrays
65.   for (int i = 0; i < N_SAMPLES; i++) {
66.     data[i] = 0;
67.     scaledData[i] = GRAPH_TOP + GRAPH_HEIGHT / 2;
68.   }
69.
70.   previousY = GRAPH_TOP + GRAPH_HEIGHT / 2;
71. }
72.
73. void loop() {
74.   // here we acquire the signal and store it in the array
75.   int adcValue = analogRead(ANALOG_PIN);
76.   data[x] = adcValue;
77.
78.   // Map the current ADC value to the graph area for real-time plotting
79.   int y = map(
80.     adcValue,
81.     0,
82.     1023,
83.     GRAPH_TOP + GRAPH_HEIGHT - 1,
84.     GRAPH_TOP
85.   );
86.
87.   scaledData[x] = y;
88.
89.   // plot the signal during acquisition
90.   if (x > 0) {
91.     tft.drawLine(
92.       x - 1,
93.       previousY,
94.       x,
95.       y,
96.       HX8357_GREEN
97.     );
98.   }
99.
100.  previousY = y;
101.  x++;
102.
103.  // This condition is reached when the graph reaches the end of the display
104.  if (x >= N_SAMPLES) {
105.    // Rescale the acquired data using the actual minimum and maximum values
106.    rescalearray(data);
107.
108.    // Clear the entire screen
109.    tft.fillScreen(HX8357_BLACK);
110.
111.    // Plot the rescaled graph
112.    plotarray(scaledData);
113.
114.    // Draw the title only after the final graph has been plotted
115.    drawText();
116.
117.    // Pause before starting a new acquisition
118.    delay(pauseAfterPlot);
```

```
119.
120.     x = 0;
121.     previousY = GRAPH_TOP + GRAPH_HEIGHT / 2;
122.     tft.fillScreen(HX8357_BLACK);
123.   }
124.
125.   delay(sampleDelay);
126. }
127.
128. // Find the maximum value in the array
129. int findmax(int data[]) {
130.   int maxValue = data[0];
131.
132.   for (int i = 1; i < N_SAMPLES; i++) {
133.     if (data[i] > maxValue) {
134.       maxValue = data[i];
135.     }
136.   }
137.
138.   return maxValue;
139. }
140.
141. // Find the minimum value in the array
142. int findmin(int data[]) {
143.   int minValue = data[0];
144.
145.   for (int i = 1; i < N_SAMPLES; i++) {
146.     if (data[i] < minValue) {
147.       minValue = data[i];
148.     }
149.   }
150.
151.   return minValue;
152. }
153.
154. // Rescale the acquired signal to fit the upper half of the screen
155. void rescalearray(int data[]) {
156.   int maxValue = findmax(data);
157.   int minValue = findmin(data);
158.
159.   // If the signal is constant, draw a flat line in the middle of the graph area
160.   if (maxValue == minValue) {
161.     for (int i = 0; i < N_SAMPLES; i++) {
162.       scaledData[i] = GRAPH_TOP + GRAPH_HEIGHT / 2;
163.     }
164.     return;
165.   }
166.
167.   // Map each acquired value between the minimum and maximum signal values
168.   for (int i = 0; i < N_SAMPLES; i++) {
169.     scaledData[i] = map(
170.       data[i],
171.       minValue,
172.       maxValue,
173.       GRAPH_TOP + GRAPH_HEIGHT - 1,
174.       GRAPH_TOP
175.     );
176.   }
177. }
178.
179. // Plot the signal stored in the given array
180. void plotarray(int data[]) {
181.   for (int i = 1; i < N_SAMPLES; i++) {
182.     tft.drawLine(
183.       i - 1,
184.       data[i - 1],
185.       i,
186.       data[i],
187.       HX8357_GREEN
188.     );
189.   }
```

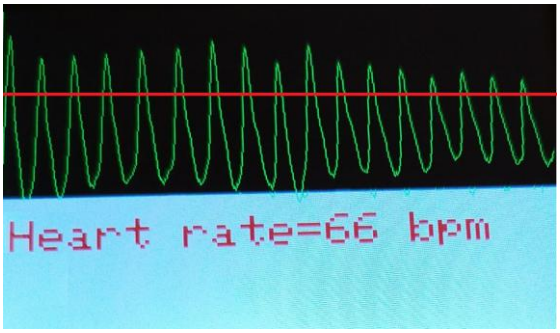
```
}

// Draw the title and background text area only on the final screen
void drawText() {
  tft.fillRect(
    0,
    GRAPH_HEIGHT,
    SCREEN_WIDTH,
    SCREEN_HEIGHT - GRAPH_HEIGHT,
    HX8357_CYAN
  );

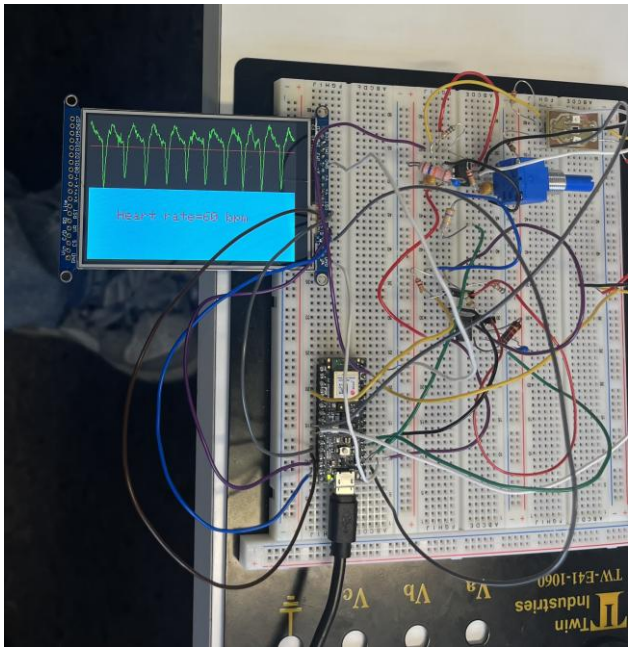
  tft.setTextColor(HX8357_RED);
  tft.setTextSize(3);
  tft.setCursor(80, TEXT_TOP);
  tft.print("Heart monitor");
}

```

4. **Optional:** From the acquired array, write a code to measure the heart rate and display the corresponding value on the display (see example in figure)



Also report picture of the working circuit and related screenshots.



```
#include <Adafruit_GFX.h>
#include <Adafruit_HX8357.h>
#include <SPI.h>

// Define the pins used to control the Adafruit display
#define TFT_CS   10
```

```
#define TFT_DC    9
#define TFT_RST  -1

#define ANALOG_PIN A0      // Analog output of the circuit
#define IR_LED_PIN 7       // IR LED control pin

// Create an instance of the Adafruit_HX8357 display object
Adafruit_HX8357 tft = Adafruit_HX8357(TFT_CS, TFT_DC, TFT_RST);

// Define display parameters
#define SCREEN_WIDTH   480
#define SCREEN_HEIGHT  320

#define GRAPH_TOP      0
#define GRAPH_HEIGHT   160     // Upper half of the screen

#define TEXT_TOP       210

// The number of samples is equal to the screen width
#define N_SAMPLES SCREEN_WIDTH

// Time interval between two samples, in milliseconds
const int sampleDelay = 25;

// Delay between the final plot and the start of a new acquisition
const int pauseAfterPlot = 5000;

// Array used to store the raw ADC values
int data[N_SAMPLES];

// Array used to store the scaled y-coordinates for plotting the signal
int scaledData[N_SAMPLES];

int x = 0;
int previousY = 0;

// Function prototypes
int findmax(int data[]);
int findmin(int data[]);
int findmean(int data[]);
void rescalearray(int data[]);
void plotarray(int data[]);
int calculateBPM(int data[]);
void drawFinalText(int bpm);
void drawThresholdLine(int rawThreshold);

void setup() {
  // Configure the IR LED pins as output
  pinMode(IR_LED_PIN, OUTPUT);
  digitalWrite(IR_LED_PIN, HIGH);

  pinMode(ANALOG_PIN, INPUT);

  // here we initialize the display
  tft.begin();
  tft.setRotation(1);

  tft.fillScreen(HX8357_BLACK);

  // here we initialize the data arrays
  for (int i = 0; i < N_SAMPLES; i++) {
    data[i] = 0;
    scaledData[i] = GRAPH_TOP + GRAPH_HEIGHT / 2;
  }

  previousY = GRAPH_TOP + GRAPH_HEIGHT / 2;
}

void loop() {
  // Acquire the signal and store it in the array
  int adcValue = analogRead(ANALOG_PIN);
```

```
  data[x] = adcValue;

  // Map the current ADC value to the graph area for real-time plotting
  int y = map(
    adcValue,
    0,
    1023,
    GRAPH_TOP + GRAPH_HEIGHT - 1,
    GRAPH_TOP
  );

  scaledData[x] = y;

  // here we plot the heart rate graph during the acquisition
  if (x > 0) {
    tft.drawLine(
      x - 1,
      previousY,
      x,
      y,
      HX8357_GREEN
    );
  }

  previousY = y;
  x++;

  // This condition is reached when the graph reaches the end of the display
  if (x >= N_SAMPLES) {
    // Calculate the heart rate from the acquired data
    int bpm = calculateBPM(data);

    // Calculate the automatic threshold as the mean value of the signal
    int threshold = findmean(data);

    // Rescale the acquired data using the actual minimum and maximum values
    rescalearray(data);

    // Clear the entire screen
    tft.fillScreen(HX8357_BLACK);

    // Plot the rescaled graph
    plotarray(scaledData);

    // Draw the red threshold line corresponding to the mean value
    drawThresholdLine(threshold);

    // Draw the final result on the display
    drawFinalText(bpm);

    // Pause before starting a new acquisition
    delay(pauseAfterPlot);

    // Reset the sample index and previous y-coordinate
    x = 0;
    previousY = GRAPH_TOP + GRAPH_HEIGHT / 2;

    // Clear the screen before starting the next live acquisition
    tft.fillScreen(HX8357_BLACK);
  }

  delay(sampleDelay);
}

// Find the maximum value in the array
int findmax(int data[]) {
  int maxValue = data[0];

  for (int i = 1; i < N_SAMPLES; i++) {
    if (data[i] > maxValue) {
      maxValue = data[i];
```

```
    }
  }

  return maxValue;
}

// Find the minimum value in the array
int findmin(int data[]) {
  int minValue = data[0];

  for (int i = 1; i < N_SAMPLES; i++) {
    if (data[i] < minValue) {
      minValue = data[i];
    }
  }

  return minValue;
}

// Calculate the mean value of the array
int findmean(int data[]) {
  long sum = 0;

  for (int i = 0; i < N_SAMPLES; i++) {
    sum += data[i];
  }

  return sum / N_SAMPLES;
}

// Rescale the acquired signal to fit the upper half of the screen
void rescalearray(int data[]) {
  int maxValue = findmax(data);
  int minValue = findmin(data);

  // If the signal is constant, draw a flat line in the middle of the graph area
  if (maxValue == minValue) {
    for (int i = 0; i < N_SAMPLES; i++) {
      scaledData[i] = GRAPH_TOP + GRAPH_HEIGHT / 2;
    }
    return;
  }

  // Map each acquired value between the minimum and maximum signal values
  for (int i = 0; i < N_SAMPLES; i++) {
    scaledData[i] = map(
      data[i],
      minValue,
      maxValue,
      GRAPH_TOP + GRAPH_HEIGHT - 1,
      GRAPH_TOP
    );
  }
}

// Plot the signal stored in the given array
void plotarray(int data[]) {
  for (int i = 1; i < N_SAMPLES; i++) {
    tft.drawLine(
      i - 1,
      data[i - 1],
      i,
      data[i],
      HX8357_GREEN
    );
  }
}

// Calculate the heart rate using the mean value of the IR signal as threshold
int calculateBPM(int data[]) {
  int minValue = findmin(data);
```

```
  int maxValue = findmax(data);
  int threshold = findmean(data);

  int amplitude = maxValue - minValue;

  // If the signal amplitude is too small, no reliable beat is detected
  if (amplitude < 10) {
    return 0;
  }

  int beatCount = 0;
  bool aboveThreshold = false;

  // Minimum distance between two beats to avoid counting the same peak twice
  int minDistanceSamples = 300 / sampleDelay;

  int lastBeatIndex = -minDistanceSamples;

  for (int i = 1; i < N_SAMPLES; i++) {
    if (data[i] > threshold && !aboveThreshold) {
      if (i - lastBeatIndex >= minDistanceSamples) {
        beatCount++;
        lastBeatIndex = i;
      }

      aboveThreshold = true;
    }

    if (data[i] < threshold) {
      aboveThreshold = false;
    }
  }

  float acquisitionTime = (N_SAMPLES * sampleDelay) / 1000.0;

  int bpm = (beatCount * 60.0) / acquisitionTime;

  return bpm;
}

// Draw the red threshold line corresponding to the mean value
void drawThresholdLine(int rawThreshold) {
  int minValue = findmin(data);
  int maxValue = findmax(data);

  if (maxValue == minValue) {
    return;
  }

  int yThreshold = map(
    rawThreshold,
    minValue,
    maxValue,
    GRAPH_TOP + GRAPH_HEIGHT - 1,
    GRAPH_TOP
  );

  tft.drawLine(
    0,
    yThreshold,
    SCREEN_WIDTH,
    yThreshold,
    HX8357_RED
  );
}

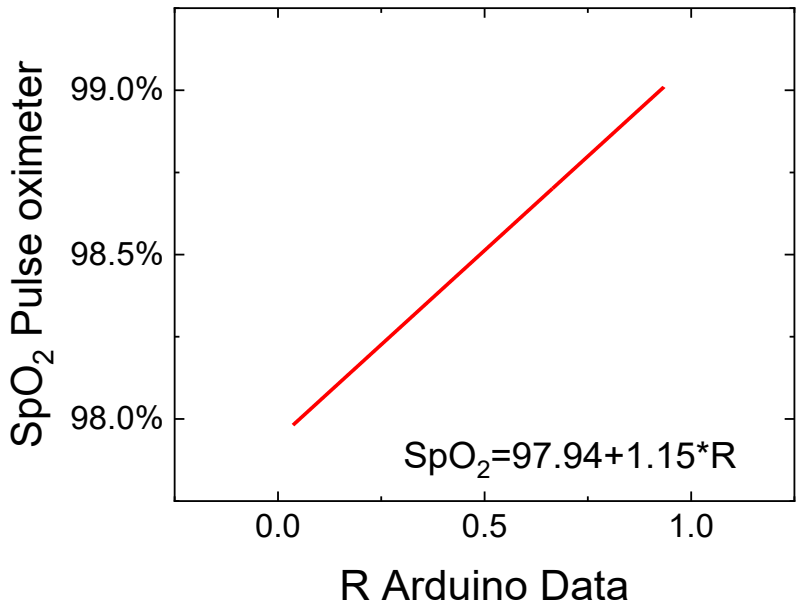
// Draw the final heart rate value on the display
void drawFinalText(int bpm) {
  tft.fillRect(
    0,
    GRAPH_HEIGHT,
```

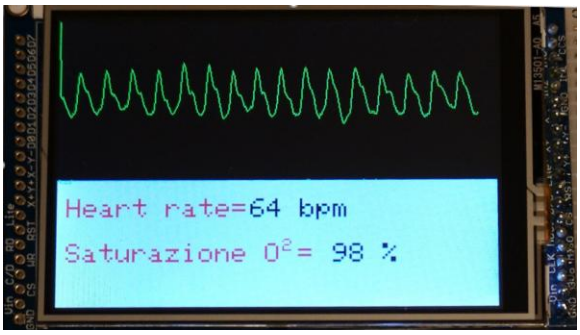
```
291.     SCREEN_WIDTH,
292.     SCREEN_HEIGHT - GRAPH_HEIGHT,
293.     HX8357_CYAN
294.   );
295.
296.   tft.setTextColor(HX8357_RED);
297.   tft.setTextSize(3);
298.   tft.setCursor(65, TEXT_TOP);
299.
300.   tft.print("Heart rate=");
301.   tft.print(bpm);
302.   tft.print(" bpm");
303. }
304.
```

5. **Optional:** Write the code to measure the oxygen concentration in the blood by evaluating the ratio *R* between the absorption of the red and infrared component, according to the following formula

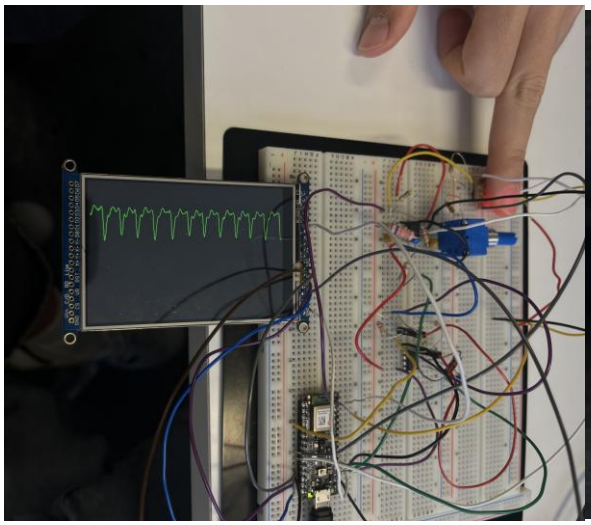
$$R = \frac{\left(\frac{AC}{DC}\right)_{RED}}{\left(\frac{AC}{DC}\right)_{IR}} = \frac{\frac{(V_{\max RED} - V_{min\ RED})}{V_{min\ RED}}}{\frac{(V_{\max IR} - V_{min\ IR})}{V_{min\ IR}}}$$

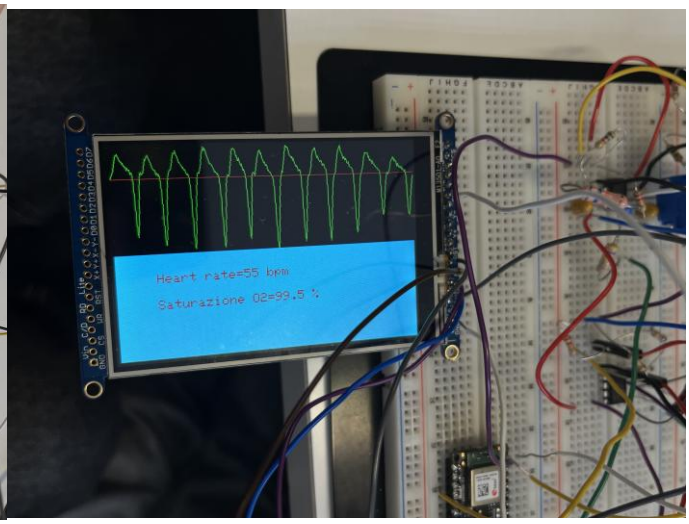
From R, the saturation level can be calculated, by using the formula below.





Also report picture of the working circuit and related screenshots.






```
#include <Adafruit_GFX.h>
#include <Adafruit_HX8357.h>
#include <SPI.h>

// Define the pins used to control the Adafruit display
#define TFT_CS   10
#define TFT_DC    9
#define TFT_RST   -1

#define ANALOG_PIN A0      // Analog output of the circuit

#define IR_LED_PIN  7      // IR LED control pin
#define RED_LED_PIN 6      // Red LED control pin

// Create an instance of the Adafruit_HX8357 display object
Adafruit_HX8357 tft = Adafruit_HX8357(TFT_CS, TFT_DC, TFT_RST);
```

```
17.
18. // Define display parameters
19. #define SCREEN_WIDTH   480
20. #define SCREEN_HEIGHT  320
21.
22. #define GRAPH_TOP      0
23. #define GRAPH_HEIGHT   160     // Upper half of the screen
24.
25. #define TEXT_TOP       190
26.
27. // The number of samples is equal to the screen width
28. #define N_SAMPLES SCREEN_WIDTH
29.
30. // Time interval between two samples, in milliseconds
31. const int sampleDelay = 25;
32.
33. // Delay between the final plot and the start of a new acquisition
34. const int pauseAfterPlot = 5000;
35.
36. // Array used to store the signal acquired with the IR LED turned on
37. int irData[N_SAMPLES];
38.
39. // Array used to store the signal acquired with the red LED turned on
40. int redData[N_SAMPLES];
41.
42. // Array used to store the scaled y-coordinates for plotting the signal
43. int scaledData[N_SAMPLES];
44.
45. int x = 0;
46. int previousY = 0;
47.
48. // Function prototypes
49. int findmax(int data[]);
50. int findmin(int data[]);
51. int findmean(int data[]);
52.
53. void rescalearray(int data[]);
54. void plotarray(int data[]);
55.
56. void acquireIRLive();
57. void acquireRED();
58.
59. int calculateBPM(int data[]);
60. float calculateSpO2(int redData[], int irData[]);
61.
62. void drawThresholdLine(int rawThreshold, int data[]);
63. void drawFinalText(int bpm, float spo2);
64.
65. void setup() {
66.   // Configure the LED pins as outputs
67.   pinMode(IR_LED_PIN, OUTPUT);
68.   pinMode(RED_LED_PIN, OUTPUT);
69.
70.   // here we configure the analog input pin
71.   pinMode(ANALOG_PIN, INPUT);
72.
73.   // Turn off both LEDs at startup
74.   digitalWrite(IR_LED_PIN, LOW);
75.   digitalWrite(RED_LED_PIN, LOW);
76.
77.   // here we initialize the display
78.   tft.begin();
79.   tft.setRotation(1);
80.
81.   tft.fillScreen(HX8357_BLACK);
82.
83.   // here we initialize the data arrays
84.   for (int i = 0; i < N_SAMPLES; i++) {
85.     irData[i] = 0;
86.     redData[i] = 0;
87.     scaledData[i] = GRAPH_TOP + GRAPH_HEIGHT / 2;
```

```
  }

  previousY = GRAPH_TOP + GRAPH_HEIGHT / 2;
}

void loop() {
  // Acquire the IR signal for the heart rate plot
  acquireIRLive();

  // Acquire the red LED signal to calculate the oxygen concentration
  acquireRED();

  // Calculate the final heart rate and oxygen concentration values
  int bpm = calculateBPM(irData);
  float spo2 = calculateSpO2(redData, irData);

  int threshold = findmean(irData);

  // Rescale and display the final IR graph
  rescalearray(irData);

  tft.fillScreen(HX8357_BLACK);

  plotarray(scaledData);

  drawThresholdLine(threshold, irData);

  drawFinalText(bpm, spo2);

  delay(pauseAfterPlot);

  // Clear the screen before starting a new measurement
  tft.fillScreen(HX8357_BLACK);

  x = 0;
  previousY = GRAPH_TOP + GRAPH_HEIGHT / 2;
}

// here we plot the heart rate graph during the acquisition
void acquireIRLive() {
  digitalWrite(RED_LED_PIN, LOW);
  digitalWrite(IR_LED_PIN, HIGH);

  delay(100);

  tft.fillScreen(HX8357_BLACK);

  x = 0;
  previousY = GRAPH_TOP + GRAPH_HEIGHT / 2;

  while (x < N_SAMPLES) {
    int adcValue = analogRead(ANALOG_PIN);
    irData[x] = adcValue;

    int y = map(
      adcValue,
      0,
      1023,
      GRAPH_TOP + GRAPH_HEIGHT - 1,
      GRAPH_TOP
    );

    scaledData[x] = y;

    if (x > 0) {
      tft.drawLine(
        x - 1,
        previousY,
        x,
        y,
        HX8357_GREEN
```

```
      );
    }

    previousY = y;
    x++;

    delay(sampleDelay);
  }

  digitalWrite(IR_LED_PIN, LOW);
}

// Acquire the signal with the red LED turned on
// This acquisition is used for SpO2 calculation
void acquireRED() {
  digitalWrite(IR_LED_PIN, LOW);
  digitalWrite(RED_LED_PIN, HIGH);

  delay(100);

  for (int i = 0; i < N_SAMPLES; i++) {
    redData[i] = analogRead(ANALOG_PIN);
    delay(sampleDelay);
  }

  digitalWrite(RED_LED_PIN, LOW);
}

// Find the maximum value in the array
int findmax(int data[]) {
  int maxValue = data[0];

  for (int i = 1; i < N_SAMPLES; i++) {
    if (data[i] > maxValue) {
      maxValue = data[i];
    }
  }

  return maxValue;
}

// Find the minimum value in the array
int findmin(int data[]) {
  int minValue = data[0];

  for (int i = 1; i < N_SAMPLES; i++) {
    if (data[i] < minValue) {
      minValue = data[i];
    }
  }

  return minValue;
}

// Calculate the mean value of the array
int findmean(int data[]) {
  long sum = 0;

  for (int i = 0; i < N_SAMPLES; i++) {
    sum += data[i];
  }

  return sum / N_SAMPLES;
}

// Rescale the acquired signal to fit the upper half of the screen
void rescalearray(int data[]) {
  int maxValue = findmax(data);
  int minValue = findmin(data);

  // If the signal is constant, draw a flat line in the middle of the graph area
```

```
  if (maxValue == minValue) {
    for (int i = 0; i < N_SAMPLES; i++) {
      scaledData[i] = GRAPH_TOP + GRAPH_HEIGHT / 2;
    }
    return;
  }

  // Map each acquired value between the minimum and maximum signal values
  for (int i = 0; i < N_SAMPLES; i++) {
    scaledData[i] = map(
      data[i],
      minValue,
      maxValue,
      GRAPH_TOP + GRAPH_HEIGHT - 1,
      GRAPH_TOP
    );
  }
}

// Plot the signal stored in the given array
void plotarray(int data[]) {
  for (int i = 1; i < N_SAMPLES; i++) {
    tft.drawLine(
      i - 1,
      data[i - 1],
      i,
      data[i],
      HX8357_GREEN
    );
  }
}

// Calculate the heart rate using the mean value of the IR signal as threshold
int calculateBPM(int data[]) {
  int minValue = findmin(data);
  int maxValue = findmax(data);
  int threshold = findmean(data);

  int amplitude = maxValue - minValue;

  // If the signal amplitude is too small, no reliable beat is detected
  if (amplitude < 10) {
    return 0;
  }

  int beatCount = 0;
  bool aboveThreshold = false;

  // Minimum distance between two beats to avoid counting the same peak twice
  int minDistanceSamples = 300 / sampleDelay;

  int lastBeatIndex = -minDistanceSamples;

  for (int i = 1; i < N_SAMPLES; i++) {
    if (data[i] > threshold && !aboveThreshold) {
      if (i - lastBeatIndex >= minDistanceSamples) {
        beatCount++;
        lastBeatIndex = i;
      }

      aboveThreshold = true;
    }

    if (data[i] < threshold) {
      aboveThreshold = false;
    }
  }

  float acquisitionTime = (N_SAMPLES * sampleDelay) / 1000.0;

  int bpm = (beatCount * 60.0) / acquisitionTime;
```

```
301.
302.   return bpm;
303. }
304.
305. float calculateSpO2(int redData[], int irData[]) {
306.   int redMax = findmax(redData);
307.   int redMin = findmin(redData);
308.   int redMean = findmean(redData);
309.
310.   int irMax = findmax(irData);
311.   int irMin = findmin(irData);
312.   int irMean = findmean(irData);
313.
314.   int acRed = redMax - redMin;
315.   int dcRed = redMean;
316.
317.   int acIR = irMax - irMin;
318.   int dcIR = irMean;
319.
320.   if (acRed <= 0 || acIR <= 0 || dcRed <= 0 || dcIR <= 0) {
321.     return 0;
322.   }
323.
324.   float ratioRed = acRed / dcRed;
325.   float ratioIR = acIR / dcIR;
326.
327.   if (ratioIR <= 0) {
328.     return 0;
329.   }
330.
331.   float R = ratioRed / ratioIR;
332.
333.   // Apply the SpO2 formula used in the assignment
334.   float spo2 = 97.94 + 1.15 * R;
335.
336.   if (spo2 > 100) {
337.     spo2 = 100;
338.   }
339.
340.   if (spo2 < 0) {
341.      spo2 = 0;
342.   }
343.
344.   return spo2;
345. }
346.
347. // Draw the red threshold line corresponding to the mean value
348. void drawThresholdLine(int rawThreshold, int data[]) {
349.   int minValue = findmin(data);
350.   int maxValue = findmax(data);
351.
352.   if (maxValue == minValue) {
353.     return;
354.   }
355.
356.   int yThreshold = map(
357.     rawThreshold,
358.     minValue,
359.     maxValue,
360.     GRAPH_TOP + GRAPH_HEIGHT - 1,
361.     GRAPH_TOP
362.   );
363.
364.   tft.drawLine(
365.     0,
366.     yThreshold,
367.     SCREEN_WIDTH,
368.     yThreshold,
369.     HX8357_RED
370.   );
371. }
```

```
372.
373. // Draw the final heart rate and oxygen saturation values
374. void drawFinalText(int bpm, float spo2) {
375.   tft.fillRect(
376.     0,
377.     GRAPH_HEIGHT,
378.     SCREEN_WIDTH,
379.     SCREEN_HEIGHT - GRAPH_HEIGHT,
380.     HX8357_CYAN
381.   );
382.
383.   tft.setTextColor(HX8357_RED);
384.   tft.setTextSize(2);
385.
386.   tft.setCursor(60, TEXT_TOP);
387.   tft.print("Heart rate=");
388.   tft.print(bpm);
389.   tft.print(" bpm");
390.
391.   tft.setCursor(60, TEXT_TOP + 40);
392.   tft.print("Saturazione O2=");
393.   tft.print(spo2, 1);
394.   tft.print(" %");
395. }
396.
```

LINK FOR THE LABORATORY VIDEOS:
https://drive.google.com/drive/folders/10YvJ3GA_eYZW2PBrk3dZOL_iGaGEAMxm?usp=share_link